

Day 9 Homework

Mithran Sadasivam. My code is in my GitHub repo, in the folder **HW #9**:
<https://github.com/mithransadasivam/FDE>

1. Edge cases

I wrote 5 browser tests in **tests/e2e/e2e_edge_cases.py**. Each test waits until the answer is finished before it checks anything, and it checks the shape of the page, not the exact words.

The 3 from the slide:

1. Empty question: nothing gets sent. I send an empty question and then a real one, and check there are only 2 messages.
2. Very long question: I pasted a fake VPN error log of about 3,000 characters. I check I get an answer or a decline and no error box.
3. A question the documents can't answer (capital of Australia): the decline line shows and there is no Sources line.

My own 2:

4. One emoji: I check there is no error box. People paste strange stuff, and an emoji could break the code.
5. Reload: I ask two questions, reload the page, and check the chat is empty again. Everyone reloads pages.

Local runs: with the fake chatbot 5 passed. With the real chatbot 5 passed too (about 95 seconds).

```
"""Day 9 homework: edge-case browser tests for the streaming chat app.
```

```
Run them with:                pytest tests/e2e/e2e_edge_cases.py
Without models (as in CI):    CHAT_FAKE=1 pytest tests/e2e/e2e_edge_cases.py
```

```
Each test checks what must always be true (structure), never the exact wording of an answer.
Whenever a question is sent, the test waits for the "Sources:" or decline line, which the app
shows only when the answer is complete.
"""
```

```
from playwright.sync_api import Page, expect
```

```
from tests.e2e.helpers import CHAT_BOX, DECLINED, SOURCES, TIMEOUT, ask, messages
```

```
APP_FILE = "app/rag_app_stream.py"
```

```
FINISHED = (SOURCES, DECLINED) # either line means the answer is complete
```

```
def wait_for_end_of_answer(page: Page, count: int = 1) -> None:
    """Wait until `count` answers have finished (each ends with a Sources or decline line)."""
    done = page.get_by_text(SOURCES).or_(page.get_by_text(DECLINED))
    expect(done).to_have_count(count, timeout=TIMEOUT)
```

```
# --- The three edge cases from the curriculum -----
```

```
def test_an_empty_question_is_not_sent(page: Page, app_url: str):
    # An empty question, then a real one. If the empty one had been sent, there would be 4 messages.
    ask(page, app_url, "")
    ask(page, app_url, "How often are laptops replaced?", open_page=False)
    wait_for_end_of_answer(page)
    expect(messages(page)).to_have_count(2) # only the real question and its answer
    expect(page.get_by_placeholder(CHAT_BOX)).to_be_enabled()
```

```
def test_a_very_long_question_gets_an_answer_or_a_decline_and_no_error(page: Page, app_url: str):
    pasted_log = "ERROR vpn-client connection reset by peer (retrying) " * 55 # about 3,000 characters
    assert 2_900 < len(pasted_log) < 3_100
    ask(page, app_url, pasted_log + " Why does my VPN keep disconnecting?")
    wait_for_end_of_answer(page)
    expect(page.get_by_test_id("stException")).to_have_count(0)
    expect(messages(page)).to_have_count(2)
```

```
def test_a_question_no_document_answers_is_declined_without_sources(page: Page, app_url: str):
```

```

ask(page, app_url, "What is the capital city of Australia?")
expect(page.get_by_text(DECLINED)).to_be_visible(timeout=TIMEOUT)
expect(page.get_by_text(SOURCES)).to_have_count(0)

# --- My two edge cases -----
# Why these two: users do paste odd characters (an emoji can break encoding or embedding code),
# and every real user reloads the page; stale or half-saved chat state is an easy bug to miss.

def test_a_single_emoji_gives_no_error(page: Page, app_url: str):
    ask(page, app_url, "\N{THUMBS UP SIGN}")
    wait_for_end_of_answer(page) # a decline is fine
    expect(page.get_by_test_id("stException")).to_have_count(0)
    expect(page.get_by_placeholder(CHAT_BOX)).to_be_enabled()

def test_reloading_the_page_clears_the_chat(page: Page, app_url: str):
    ask(page, app_url, "How long can a VPN session stay connected?")
    wait_for_end_of_answer(page, 1)
    ask(page, app_url, "What is the password policy?", open_page=False)
    wait_for_end_of_answer(page, 2)
    expect(messages(page)).to_have_count(4)

    page.reload()
    expect(page.get_by_placeholder(CHAT_BOX)).to_be_visible(timeout=TIMEOUT)
    expect(messages(page)).to_have_count(0)

```

2. CI (the green fast job)

I added the new test file to the step "Browser tests with the fake chatbot" in the workflow, so it runs on every push. Green fast job with all five tests:

<https://github.com/mithransadasivam/FDE/actions/runs/37714031080>

It did not go green the first time. Two things were wrong:

- **stream_answer** was still unfinished from Day 9 Activity 2, so 5 unit tests failed. I finished it.
- **rank-bm25** was missing from requirements.txt. It was installed on my laptop but not on GitHub's clean machine, so the tests couldn't even start. I added it.

3. Red, then green

I made a branch called **break-the-decline** and removed the line that shows "No sources: the bot declined."

Pull request: <https://github.com/mithransadasivam/FDE/pull/2>

Red run (it failed): <https://github.com/mithransadasivam/FDE/actions/runs/37714217839>

Error line from the log: AssertionError: Locator expected to be visible ... Error: element(s) not found ... waiting for get_by_text("No sources: the bot declined."). Result: 3 failed, 5 passed.

The 3 tests that failed: the streamed decline test, my "capital of Australia" test, and my emoji test. The bot still said "I don't know", but the decline line was gone, so all three waited for it and timed out.

Then I put the line back and pushed to the same branch. It went green:

<https://github.com/mithransadasivam/FDE/actions/runs/37714738335>

I closed the pull request without merging. The break only ever lived on the branch, so main stayed green.

4. Reviewing a test an agent wrote

I picked scenario 4 from specs/chat-app.md (two questions in a row). One honest note: the playwright-test-generator agent wasn't loaded in my Claude Code session, so another Claude agent wrote the test from the same spec and rules. It was not recorded against the live app.

What I found when I checked it:

1. **It was flaky.** I ran it 3 times. The first run failed 3 of 4 tests because the app was still starting up and the default 5 second wait was too short. The other 2 runs passed. Fix: I raised the wait to 20 seconds in playwright.config.ts.

2. **One check was too loose.** It only counted the Sources lines on the whole page, so it never proved each answer had its own. Fix: it now checks every answer for its own [n] citation and its own Sources line.

The rest was fine: no exact-wording checks, it waits for the Sources line before asking the next question, no fixed waits, and the locators are stable.

After the fixes it passed 4 of 4, three times in a row.

Test as the agent wrote it:

```
test('Two questions in a row build up the conversation', async ({ page }) => {
  // 1. Open the app, ask the VPN question, wait for "Sources: ".
  await page.goto('/');
  const box = page.getByPlaceholder(CHAT_BOX);
  await box.fill('How long can a VPN session stay connected?');
  await box.press('Enter');
  const sources = page.getByText(/^Sources: /);
  await expect(sources).toHaveCount(1);

  // 2. Without reloading, ask "How often are laptops replaced?" and wait for the second "Sources: ".
  await box.fill('How often are laptops replaced?');
  await box.press('Enter');
  await expect(sources).toHaveCount(2);

  // Four chat messages in order: question 1, answer 1, question 2, answer 2.
  const messages = page.getByTestId('stChatMessage');
  await expect(messages).toHaveCount(4);
  await expect(messages.nth(0)).toContainText('How long can a VPN session stay connected?');
  await expect(messages.nth(2)).toContainText('How often are laptops replaced?');

  // Each answer has its own [n] citation.
  await expect(messages.nth(1)).toContainText(/\[d+\]/);
  await expect(messages.nth(3)).toContainText(/\[d+\]/);
});
```

My fixed version:

```
test('Two questions in a row build up the conversation', async ({ page }) => {
  // 1. Open the app, ask the VPN question, wait for "Sources: ".
  await page.goto('/');
  const box = page.getByPlaceholder(CHAT_BOX);
  await box.fill('How long can a VPN session stay connected?');
  await box.press('Enter');
  const sources = page.getByText(/^Sources: /);
  await expect(sources).toHaveCount(1);

  // 2. Without reloading, ask "How often are laptops replaced?" and wait for the second "Sources: ".
  await box.fill('How often are laptops replaced?');
  await box.press('Enter');
  await expect(sources).toHaveCount(2);

  // Four chat messages in order: question 1, answer 1, question 2, answer 2.
  const messages = page.getByTestId('stChatMessage');
  await expect(messages).toHaveCount(4);
  await expect(messages.nth(0)).toContainText('How long can a VPN session stay connected?');
  await expect(messages.nth(2)).toContainText('How often are laptops replaced?');

  // Each answer has its own [n] citation and its own Sources line.
  for (const answer of [messages.nth(1), messages.nth(3)]) {
    await expect(answer).toContainText(/\[d+\]/);
    await expect(answer.getByText(/^Sources: /)).toHaveCount(1);
  }
});
```